TECHSERVICE HUB

Product Requirements Document

Two-Founder Digital Services Marketplace

"Your Ideas. Our Expertise. One Digital Solution."

Version	1.0
Date	November 1, 2026
Status	**LIVE — IN PRODUCTION**
Authors	Founder A (Engineering) · Founder B (Design & Docs)
Stack	Next.js · Express · Neon PostgreSQL · Prisma · Firebase Auth · Vercel

Table of Contents

1. Product Overview 3
2. Goals & Success Metrics 3
3. Users & Roles 4
4. Scope — v1 4
5. Functional Requirements 5
6. Order Lifecycle (7-Stage Pipeline) 8
7. Web Page Map 9
8. Data Model Summary 10
9. API Summary 11
10. Integrations 12
11. Non-Functional Requirements 12

12. Known Gaps & Future Work 13

Appendix A — Environments & Demo Accounts 13

1. Product Overview

TechService Hub (TSH) is a two-founder IT & digital-services marketplace for the Bangladesh market. Customers browse a curated service catalog (websites, software projects, UI/UX, pitch decks, documentation, e-commerce, IT consulting), configure a service with priced options, choose which founder should lead the project, pay manually via local payment rails (bKash / Nagad / Rocket / bank transfer), and track the order through a transparent seven-stage delivery pipeline until files are delivered through the dashboard.

The two founders fulfill all work and administer the platform: they review submissions, verify payments, execute projects, upload deliverables, handle revision requests, and manage site content — no third-party workforce.

Product Principles

- **Direct access to the experts** — customers pick the founder whose expertise matches the project; no account managers.
- **Transparent milestone tracking** — every order exposes a seven-stage pipeline with a full status audit trail.
- **Server-authoritative trust** — roles, order state transitions, and pricing are computed and enforced exclusively server-side.
- **Snapshot integrity** — catalog edits never mutate historical orders; price/name/option snapshots are frozen at checkout.

2. Goals & Success Metrics

Goal	Metric	Target (launch + 90 days)
Convert catalog browsing into paid orders	Checkout conversion rate	≥ 2% of unique visitors
Fast payment confirmation	Median time: payment submitted → verified	< 2 hours
Reliable delivery	Orders delivered by agreed deadline	≥ 90%
Customer satisfaction	Average review rating on delivered orders	≥ 4.5 / 5
Healthy revision load	Revision requests per delivered order	< 1.5

3. Users & Roles

Role	Who	Capabilities
CUSTOMER	Buyers of services	Browse catalog, manage cart, checkout, submit payments, track orders, request revisions, download deliverables, review completed orders, contact support
ADMIN	The two co-founders	Everything a customer can do, plus: stats dashboard, order management & assignment, deliverable uploads, payment verification, revision responses, customer management, message inbox, review curation, homepage CMS

There are exactly two roles (`CUSTOMER` | `ADMIN`). Roles are written only by provisioning scripts / DB — never accepted from client traffic.

Founder Model

Admin users carry a **FounderProfile** — the public identity customers see. Each founder has a discipline that gates which catalog services they can lead:

Founder	Discipline	Covers
Founder A	`WEB_SOFTWARE_ECOMMERCE`	Website development, software projects, e-commerce, cloud deployment
Founder B	`DESIGN_DOCS_UX`	Presentation design, technical documentation, UI/UX
Either	`BOTH`	IT consulting & cross-discipline services

At checkout the customer picks `FOUNDER_A`, `FOUNDER_B`, or `AUTO_ASSIGN`. The chosen founder is resolved server-side and recorded with a preference source (`CUSTOMER_CHOICE` | `AUTO_ASSIGN` | `ADMIN_OVERRIDE`); admins may reassign internally — the customer is always notified.

4. Scope — v1

In Scope

Feature Area	Delivered in v1
Authentication	Firebase email/password + Google sign-in, email verification, password reset, profile sync
Service catalog	7 categories, option groups with price/time deltas, search, sort, featured flags
Cart & checkout	Deduped cart items, option selection, founder picker, snapshot pricing, idempotent order creation
Payments	Manual transaction-ID submission (bKash, Nagad, Rocket, Bank Transfer) + admin verification
Order pipeline	7-stage status machine with enforced transitions and audit history
Files	Requirement attachments & versioned deliverables via signed Cloudinary URLs
Revisions & reviews	Customer revision requests, admin responses, 1–5★ order reviews curated into testimonials

Admin panel	Stats, orders, payments, customers, messages, review curation, homepage CMS
Content	DB-driven homepage (hero, why-us, how-it-works, FAQ), portfolio, testimonials
Notifications	In-app notification feed with read marking

Out of Scope (v1)

- Automated payment gateway integration (bKash/Nagad APIs) — schema reserves `providerRef` and `provider="MANUAL"` for a drop-in upgrade.
- Multi-vendor marketplace — fulfillment is the two founders only.
- Server-side email/SMTP — transactional email rides Firebase's built-in templates.
- Mobile native apps — responsive web only.

5. Functional Requirements

5.1 Authentication & Accounts

ID	Requirement	Priority
FR-01	Firebase email/password registration with verification email; first login lands in-app immediately for provisioned accounts	MUST
FR-02	Google sign-in via popup; verified Google identities sync automatically	MUST
FR-03	`POST /auth/sync` exchanges a Firebase ID token for a Neon profile — provisions a `CUSTOMER` row on first sight, claims pre-seeded founder rows on verified-email match inside a transaction (no self-service role escalation)	MUST
FR-04	Password reset & email verification flow via Firebase templates (`/forgot-password`, `/reset-password`)	MUST
FR-05	Profile view/update (`GET/PATCH /users/me`); roles and status are server-authoritative	MUST

5.2 Service Catalog

ID	Requirement	Priority
FR-06	Hierarchical catalog: `ServiceCategory` → `Service` → `ServiceOption` groups (scope / urgency / extras / platform)	MUST
FR-07	Each option carries a price delta and delivery-day delta; one default per group preselected	MUST
FR-08	Listing endpoints support pagination, search, category filter, sort; services expose slug pages with tagline/description/base price/delivery estimate	MUST
FR-09	Services are gated by founder discipline — only founders whose discipline matches (or `BOTH`) can be assigned	MUST
FR-10	Homepage content (hero, why-us, how-it-works, FAQ), testimonials and portfolio render from	SHOULD

DB (`SiteContent` JSON keys) — editable from admin without redeploy

5.3 Cart & Checkout

ID	Requirement	Priority
FR-11	Authenticated cart; one cart item per service (dedupe on re-add); option changes update price/time preview	MUST
FR-12	Checkout creates the order in a **single database transaction**: name/price snapshots, option snapshots, founder preference resolution, attachment claiming	MUST
FR-13	Idempotency key guards against double-submit / retry duplication of orders	MUST
FR-14	Customers may attach requirement files at checkout; uploads are staged to Cloudinary client-side with signed params, then claimed by the order	SHOULD
FR-15	Order numbers are human-readable: `TSH-YYYY-NNNNNN`	MUST

5.4 Payments (Manual Verification)

ID	Requirement	Priority
FR-16	Customer pays off-platform via **bKash / Nagad / Rocket / Bank Transfer** and submits transaction ID (+ optional screenshot) against the order	MUST
FR-17	Admin verifies (`VERIFIED`) or rejects (`REJECTED`) each payment in the admin panel; payment status gates order progression	MUST
FR-18	Schema reserves `provider` / `providerRef` so gateway integrations slot in without migration churn	COULD

5.5 Files & Deliverables

ID	Requirement	Priority
FR-19	All binary files live in Cloudinary (folders: requirements / deliveries / misc); Postgres stores references only — no blobs	MUST
FR-20	Admin uploads versioned deliverables per order; customers download via short-lived signed URLs; download counts tracked	MUST
FR-21	Platform degrades gracefully when Cloudinary env is absent (`FILE_STORAGE_UNCONFIGURED` error, not a crash)	SHOULD

5.6 Revisions, Reviews, Notifications, Contact

ID	Requirement	Priority
FR-22	Customers open revision requests during active/late stages; admins respond `IN_PROGRESS` / `RESOLVED` / `DECLINED` with a plan	MUST
FR-23	One 1–5★ review per delivered order; admins curate which reviews surface as public testimonials	SHOULD

FR-24	In-app notification feed (order events, payment results, revision replies) with read-marking endpoint	MUST
FR-25	Public contact form lands in the admin message inbox	MUST

5.7 Admin Operations

ID	Requirement	Priority
FR-26	Admin dashboard: platform stats (orders, revenue, pending payments, customers)	MUST
FR-27	Order management: list/filter, detail view, status transitions, founder (re)assignment, deliverable uploads	MUST
FR-28	Customer management and message inbox	MUST
FR-29	**Every admin action is written to an AuditLog** (actor, action, target, payload)	MUST
FR-30	Homepage CMS: GET/PUT of `SiteContent` keys	SHOULD

6. Order Lifecycle — Seven-Stage Pipeline

`SUBMITTED` → `UNDER_REVIEW` → `APPROVED` → `IN_PROGRESS` → `QUALITY_REVIEW` → `COMPLETED` → `DELIVERED`, with terminal off-track states `REJECTED` and `CANCELLED`.

Stage	Meaning	May transition to
SUBMITTED	Order placed, awaiting founder review	UNDER_REVIEW, REJECTED, CANCELLED
UNDER_REVIEW	Requirements being assessed	APPROVED, REJECTED, CANCELLED
APPROVED	Accepted; payment being confirmed	IN_PROGRESS, REJECTED, CANCELLED
IN_PROGRESS	Work underway	QUALITY_REVIEW, CANCELLED
QUALITY_REVIEW	Founder QA before completion	COMPLETED, IN_PROGRESS (rework)
COMPLETED	Work finished, deliverables staged	DELIVERED
DELIVERED	Handed to customer (terminal, success)	—
REJECTED / CANCELLED	Terminal, off-track	—

Enforcement: transitions are validated server-side (`assertTransition` against the `ORDER_TRANSITIONS` matrix in `packages/shared`) — no client can push an illegal state. Every change appends to `OrderStatusHistory` (audit trail surfaced as the order tracker UI).

- **Customer cancellation** is permitted only while in SUBMITTED or UNDER_REVIEW.
- **Revision requests** are permitted in IN_PROGRESS, QUALITY_REVIEW and DELIVERED.
- The customer-facing order tracker visualizes the seven stages with timestamps.

7. Web Page Map (Next.js)

Group	Route	Purpose

Public site	/	Homepage — DB-driven hero, why-us, how-it-works, FAQ, testimonials, portfolio
	/services, /services/[slug]	Catalog listing and configurable service detail
	/cart, /checkout	Cart management; checkout with founder picker & file staging
	/contact	Contact form ⟶ admin inbox
Auth	/login	Email/password + Google sign-in
	/register	Account creation with verification email
	/forgot-password, /reset-password	Firebase-hosted reset flow completion
	—	Email verification landing / resend
Customer	/orders, /orders/[id]	Order list; detail with 7-stage tracker, payments, revisions, deliverables
	/dashboard	Customer overview
	/profile	Profile & password management
	/notifications	Notification feed
Admin	—	Role guards protect customer & admin areas (guards.tsx)
	/admin	Stats dashboard
	/admin/orders, /admin/orders/[id]	Order pipeline management, deliverables, status transitions
	/admin/payments	Verify / reject submitted payments
	/admin/customers	Customer accounts
	/admin/messages	Contact-message inbox
	—	Admin layout enforces role === ADMIN before render

8. Data Model Summary (Prisma / Neon PostgreSQL)

Model	Purpose
User	Account joined to Firebase via firebaseUid; role & status server-authoritative
FounderProfile	Public founder identity: display name, title, bio, discipline, skills, ordering, selectability
ServiceCategory / Service	Catalog tree; services carry base price (Decimal 12,2), delivery days, discipline gate, featured flag
ServiceOption	Priced options per service (group key, price/days delta, default flag, display order)
Cart / CartItem / CartItemOption	Authenticated cart; unique per (cart, service); selected option rows
Order	Order number TSH-YYYY-NNNNNN, assigned/preferred founder, preference source, idempotency key

OrderItem / OrderItemOptionSnapshot	Frozen price/name/option snapshots at checkout — immune to later catalog edits
OrderAttachment	Cloudinary reference rows; nullable orderId supports staged pre-checkout uploads
Deliverable	Versioned delivery files with download counts
RevisionRequest	Customer revision threads with admin responses
OrderStatusHistory	Full status audit trail powering the order tracker
Payment	Manual payment submissions (method, transaction ID, screenshot); provider field reserved for gateways
Notification	Per-user in-app notification feed
Review	One per order; curation flag controls public testimonial display
ContactMessage	Public contact-form inbox
AuditLog	Actor/action/target record of every admin mutation
SiteContent	JSON key/value homepage CMS blocks
PortfolioProject / Testimonial	Curated showcase content

Conventions: UUID primary keys; money as Decimal serialized to strings (no float drift); binary files never stored in Postgres; shared constants/types flow from @tsh/shared to both apps.

9. API Summary (Express, mounted under /api)

Group	Endpoints	Auth
Health	GET /health — DB + Firebase status probe	public
Auth	POST /auth/sync (token → profile), GET /auth/me	Firebase Bearer token
Catalog	GET /services, GET /services/:slug, GET /categories, GET /public/homepage	public
Users	GET /users/me, PATCH /users/me	authenticated
Cart	GET /cart, POST /cart/items, PATCH /cart/items/:id, DELETE /cart/items/:id	authenticated
Orders	POST /orders (checkout), GET /orders, GET /orders/:id, POST /orders/:id/cancel, POST /orders/:id/payments, POST /orders/:id/revisions, POST /orders/:id/review, POST /orders/:id/attachments	authenticated + ownership
Uploads	POST /uploads/sign — signed Cloudinary upload params	authenticated
Contact	POST /contact	public (rate-limited)
Notifications	GET /notifications, POST /notifications/read	authenticated
Admin	stats · orders (list/detail/status/assign/deliverables) · payments (list/verify/reject) · revisions (respond) · customers · founders · messages · review curation · site-content GET/PUT	ADMIN (audited)

Every response uses a uniform envelope ({success, data | error}) with machine-readable error codes; every route

body is zod-validated.

10. Integrations

Service	Role	Notes
Firebase Authentication	Identity provider (email/password + Google)	API verifies ID tokens with Firebase Admin; `firebaseUid` joins to Postgres; email templates for verify/reset; credential via base64 service account
Neon PostgreSQL	Primary datastore	Pooled connection string for runtime + direct URL for migrations; Prisma engines bundled for Vercel's rhel-openssl-3.0.x
Cloudinary	File storage & delivery	Signed client uploads + short-lived signed delivery URLs; graceful degrade when unconfigured
Vercel	Hosting (web + API)	Web: Next.js. API: Express in serverless function via catch-all rewrite

11. Non-Functional Requirements

Security

- CORS allowlist (production origins only); Bearer-token auth — no cookies, `credentials: false`.
- Server-side role gating (`authenticate` + `requireAdmin` middleware); roles never accepted from clients.
- Per-order ownership checks — customers can only see their own orders/attachments.
- zod validation on every route; separate stricter rate limiter on auth/contact endpoints.
- AuditLog on all admin mutations; uniform error envelope without stack-trace leakage.

Architecture & Deployment

- npm-workspaces monorepo: `apps/web`, `apps/api`, `packages/shared`; shared package is vendored into each app for serverless builds.
- Node ≥ 24; TypeScript throughout; Prisma migrations with explicit binary targets for Vercel.
- Web env: `NEXT_PUBLIC_API_URL` · API env: `CORS_ORIGINS` (mirrored pair).
- Vitest suites for API (app, env, domain, money) and shared package.

12. Known Gaps & Future Work

Item	Detail	Priority
Google provider enablement	Google sign-in provider must be toggled on in Firebase Console (API cannot enable it); web domains already authorized	HIGH

Cloudinary credentials	Production env for `CLOUDINARY_*` keys must be set before file uploads work end-to-end	HIGH
Founder identity content	Placeholder names/photos/bios in seed — replace with real founder identities pre-launch	MEDIUM
Mobile Google sign-in	`signInWithPopup` is blocked on some mobile browsers → add `signInWithRedirect` fallback	MEDIUM
Account linking UX	Same-email password + Google sign-in collision (`auth/account-exists-with-different-credential`) needs friendly handling	MEDIUM
Payment gateway	bKash/Nagad API integration to replace manual transaction-ID verification	POST-V1

Appendix A — Environments & Demo Accounts

Environment	URL
Web (production)	`https://tsh-web-sandy.vercel.app` · `https://tsh-web-rahan-hexa.vercel.app`
API (production)	`https://tsh-api-nine.vercel.app/api` · `https://tsh-api-rahan-hexa.vercel.app/api`

Account	Role	Purpose
`founder.a@techservicehub.test`	ADMIN	Founder A — engineering lead
`founder.b@techservicehub.test`	ADMIN	Founder B — design/docs lead
`customer@techservicehub.test`	CUSTOMER	Demo buyer journey

> Provisioned via `apps/api/scripts/provision-accounts.ts` (Firebase user + Neon row, email-verified). Passwords are documented privately; rotate before public launch.